



## LABORATORY 2

# Your first ROS 2 subscriber

Discover a topic and its message interface, study a depth logger, then write an odometry logger that records x, y and z.

| Course       | INRO - Instrumentation, Marine Robotics and Power Systems |
|--------------|-----------------------------------------------------------|
| Professor    | Daniel Mihai Toma                                         |
| Prerequisite | Laboratory 1 and a working Dev Container                  |
| Main result  | A student-written odometry subscriber and CSV data        |

One new concept

This laboratory focuses on the subscriber. Vehicle motion uses the terminal commands already introduced in Laboratory 1.

# 1. What you will learn

- identify a ROS 2 topic and determine its message type;
- inspect the fields of a message with `ros2 interface show`;
- connect a topic to a Python callback with `create_subscription`;
- read data from the callback message and write it to CSV;
- adapt a depth example to create an odometry logger for x, y and z.

## The communication model

| Part            | Meaning in this laboratory                             |
|-----------------|--------------------------------------------------------|
| Publisher       | The simulator produces depth and odometry messages.    |
| Topic           | A named channel such as <code>/inro/depth</code> .     |
| Message type    | The agreed structure of the data on that topic.        |
| Subscriber node | Your Python program receives the messages.             |
| Callback        | The function ROS calls whenever a new message arrives. |

### Suggested video

[YouTube: ROS 2 topic and interface inspection](#)

Use the same sequence shown in the video: list topics, obtain the type, inspect the interface, echo a message, then write code.

The official ROS 2 documentation uses this same discovery sequence because publishers and subscribers must use the same message type.

## 2. Start the simulator and discover the depth topic

In the first Dev Container terminal, start the simulator:

```
cd /workspaces/INRO-Marine-Robotics/ros2_ws
python3 src/inro_rov_demo/scripts/prepare_model.py
colcon build --packages-select inro_rov_demo --symlink-install
source install/setup.bash
ros2 launch inro_rov_demo demo.launch.py
```

Keep it running. In a second terminal, source the workspace and inspect the topic before opening the Python example:

```
cd /workspaces/INRO-Marine-Robotics/ros2_ws
source install/setup.bash

ros2 topic list
ros2 topic list -t
ros2 topic info /inro/depth
ros2 topic type /inro/depth
ros2 interface show std_msgs/msg/Float64
ros2 topic echo /inro/depth --once
```

| Question                                                                                                  | Answer from the terminal |
|-----------------------------------------------------------------------------------------------------------|--------------------------|
| What is the topic name?                                                                                   | /inro/depth              |
| What is its message type?                                                                                 | std_msgs/msg/Float64     |
| Which field contains the value?                                                                           | float64 data             |
| How will Python access it?                                                                                | msg.data                 |
| <b>Rule</b><br>Do not guess a message structure. Ask ROS for the topic type, then inspect that interface. |                          |

### 3. Worked example: depth subscriber

Open `lab02/depth_logger.py`. Its subscription connects the depth topic to a callback:

```
self.subscription = self.create_subscription(
    Float64,
    '/inro/depth',
    self.depth_callback,
    10,
)
```

Each new message makes ROS call this function. The value is read with `msg.data` and saved with the elapsed time:

```
def depth_callback(self, msg):
    elapsed = (self.get_clock().now() - self.start_time).nanoseconds / 1e9
    depth = msg.data
    self.writer.writerow([f'{elapsed:.3f}', f'{depth:.4f}'])
    self.csv_file.flush()
```

Run the example:

```
python3 src/inro_rov_demo/lab02/depth_logger.py
```

In a third sourced terminal, move down and up:

```
ros2 run inro_rov_demo drive down --seconds 4 --power 0.4
ros2 run inro_rov_demo drive up --seconds 4 --power 0.4
```

Stop the logger with `Ctrl+C` and open `lab02/depth_log.csv`. Confirm that it contains the columns `time_s`, `depth_m`.

Read the complete example

Locate the import, node class, subscription, callback, CSV header, `rc_lpy.spin(node)`, and the cleanup executed after `Ctrl+C`.

## 4. Student task: odometry subscriber

Create `lab02/odometry_logger.py`. Use the depth logger as a pattern, but inspect the new message first:

```
ros2 topic info /inro/odometry
ros2 topic type /inro/odometry
ros2 interface show nav_msgs/msg/Odometry
ros2 topic echo /inro/odometry --once
```

Follow the nested interface until you find the position fields. Your callback will need:

```
position = msg.pose.pose.position
x = position.x
y = position.y
z = position.z
```

### Program requirements

- import `Odometry` from `nav_msgs.msg`;
- subscribe to `/inro/odometry` using that message type;
- print `x`, `y` and `z` while the program runs;
- create `odometry_log.csv` with `time_s, x_m, y_m, z_m`;
- write one row for every received odometry message;
- close the file correctly after `Ctrl+C`.

| Depth example                                                                                                                                                                      | Your odometry version                         |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| Float64                                                                                                                                                                            | Odometry                                      |
| <code>/inro/depth</code>                                                                                                                                                           | <code>/inro/odometry</code>                   |
| <code>msg.data</code>                                                                                                                                                              | <code>msg.pose.pose.position.x, .y, .z</code> |
| <code>time_s, depth_m</code>                                                                                                                                                       | <code>time_s, x_m, y_m, z_m</code>            |
| <b>Learning objective</b><br>You receive the complete depth solution. Your task is to transfer the same node, callback and CSV pattern to a different topic and message interface. |                                               |

## 5. Record the experiment and prepare the report

Restart the simulator so the vehicle begins from the same pose. Start your odometry logger, then run these commands one at a time in a third sourced terminal:

```
ros2 run inro_rov_demo drive down --seconds 4 --power 0.4
ros2 run inro_rov_demo drive forward --seconds 4 --power 0.4
ros2 run inro_rov_demo drive up --seconds 4 --power 0.4
```

Allow the vehicle to settle briefly after each command. Stop the logger with Ctrl+C and verify that `odometry_log.csv` contains numerical values.

### Submit

- your `odometry_logger.py`;
- the recorded `odometry_log.csv`;
- plots of x, y and z versus time, produced with any language or application;
- a short explanation of which position changes during down, forward and up;
- answers to the five questions below.

### Questions

- 1 What message type is used by `/inro/odometry`?
- 2 Which command reveals every field in that message interface?
- 3 What event causes your callback to run?
- 4 Why is z negative underwater while depth is positive?
- 5 Why does the ROV continue moving briefly after a drive command ends?

#### Plotting

The laboratory does not prescribe a plotting language. Use Python, MATLAB, Excel, LibreOffice Calc or another tool, provided the axes, units and legend are clear.

Simulation only: never connect these teaching commands to the real ROV.